

— MINI-HACKATHON

Building GenAI Applications

Design. Build. Ship in 4 hours.



AI Tech School

4h

to build

8min

to present

15min

office hours / team

Welcome.

You have one afternoon to design and build a working GenAI system, then 8 minutes to defend your choices.

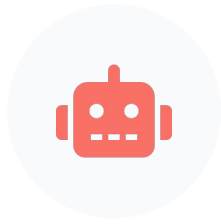
Pick a GenAI system. Design it. Build it.

Your team chooses one of the system types below. Whichever you pick, the goal is the same: a working prototype plus a clear design story.



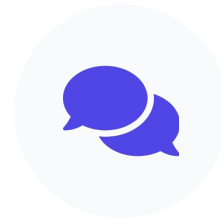
RAG

Retrieval-Augmented Generation



Agent

Tools + planning loop



Chatbot

Conversational with memory



Combined

Two or more, glued together

The next four slides go one level deeper into each.

Ground an LLM in your own documents

The idea

Index your knowledge as vector embeddings. At query time, retrieve the most relevant chunks and pass them to the LLM as context. The model answers from your data, not from training memory.

Good ideas for this hackathon

- Internal docs Q&A over Confluence or a GitHub repo
- Support assistant grounded in product manuals or past tickets
- Research helper over a folder of PDFs with citations
- Code search over a repo with file-level context

What you actually decide

Chunking	Size, overlap, by-paragraph vs by-section
Embeddings	OpenAI, Cohere, BGE, E5 — quality vs cost
Vector store	Pinecone, Weaviate, pgvector, Chroma, FAISS
Retrieval	Top-k, hybrid (BM25 + dense), reranking
Prompt design	How context is injected, citation format
Evaluation	Faithfulness, answer relevance, recall

An LLM that uses tools to get things done

The idea

The model is given a goal and a set of tools (functions it can call). It plans, calls a tool, reads the result, and loops until the goal is met or a stop condition fires. The output is not just text — it is actions taken in the world.

Good ideas for this hackathon

- Email triage agent: read inbox, classify, draft replies
- Data analyst: NL question → SQL → chart → summary
- Repo refactor bot: find a pattern across files and fix it
- Travel planner that searches, compares, and books a slot

What you actually decide

Tool set	Which tools, with what schemas and limits
Loop control	Max steps, budget, when to stop or escalate
Memory	Scratchpad, summary, short-term vs long-term
Framework	LangGraph, native function calling, custom
Safety	Confirm before destructive actions, allow-lists
Observability	Trace every tool call — LangSmith, Langfuse

A multi-turn conversation, done well

The idea

A chatbot is not just ChatGPT-in-a-window. The hard parts are persona consistency, conversation memory across turns, handling off-topic or unsafe inputs, and giving the model just enough context to stay useful without blowing the context window.

Good ideas for this hackathon

- Onboarding bot for a new hire — answers HR + IT questions
- Persona character bot with consistent voice across sessions
- Sales SDR bot that qualifies a lead through 5–7 questions
- Therapy-style journaling companion with session memory

What you actually decide

System prompt	Persona, rules, format, refusals
Memory	Last N turns vs running summary vs both
State	Per-user, per-session, persisted where
Streaming	Token streaming + UI for perceived speed
Guardrails	Off-topic, PII, prompt-injection handling
Fallbacks	What happens when the model fails or stalls

Two patterns stitched into one product

The idea

Most real products are not a pure RAG or a pure agent. They combine patterns: a chatbot front end that calls an agent, an agent whose tools include RAG retrieval, a RAG system that uses an agent to choose between indexes. The interesting design work happens at the seams.

Concrete combined patterns

- Chat + RAG: support bot that cites the manual it pulled from
- Agent + RAG: research agent that queries 3 indexes and synthesizes
- Chat + Agent: "book me a meeting" — chat collects, agent acts
- Router: classifier picks RAG, agent, or pure LLM per request

Where the seams hurt

Routing	How does the system decide which path to take
Context flow	What passes between components, in what shape
Latency	Each hop adds seconds — budget them up front
Failure modes	If RAG returns nothing, does the agent loop forever
Cost	Multiple LLM calls per user request add up fast
Debuggability	Trace one user request end-to-end across hops

8 minutes. Four things. In this order.

1 What does the product do?

One sentence on the user, one on the problem, one on what your system does for them. If it takes more than 30 seconds, it's not sharp enough yet.

2 Block diagram

Boxes for components, arrows for data flow. Include the example on the next slide as a reference. Show where the LLM calls happen and where state lives.

3 Tech review — what & why

List every meaningful choice: model, framework, vector store, embeddings, hosting. For each one: what you picked, why, and one credible alternative you considered.

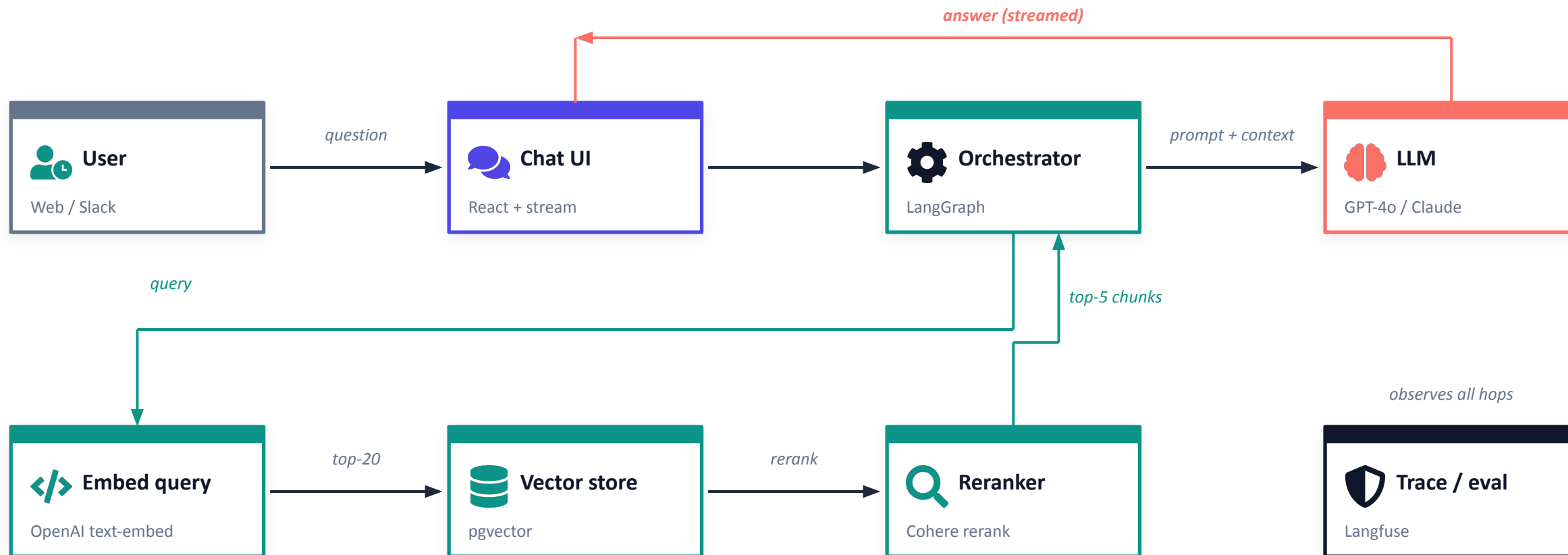
4 Issues & dilemmas

The things that did not work, the trade-offs you had to make, the open questions. This is the most interesting section. Be honest.

Rule of thumb: roughly 2 minutes per section. Demo on slide 1 if it works — talk about it on slide 4 if it doesn't.

What a clear system diagram looks like

Example: a support chatbot with RAG. Every box names a real choice. Every arrow names what flows.



I'm here. Use me well.

15 MIN / TEAM

I'll open booking slots.

Grab one when you're stuck — or when you want to pressure-test a design choice before you commit two hours to it.

First-come, first-served. Book early if you have a tricky design.

Come with a specific question, not "can you look at our project."

Come find me when...



You're stuck on architecture

Three valid ways to wire it up and you can't pick one.



Something is mysteriously slow

Latency budget blown and you don't know which hop owns it.



The model keeps doing the wrong thing

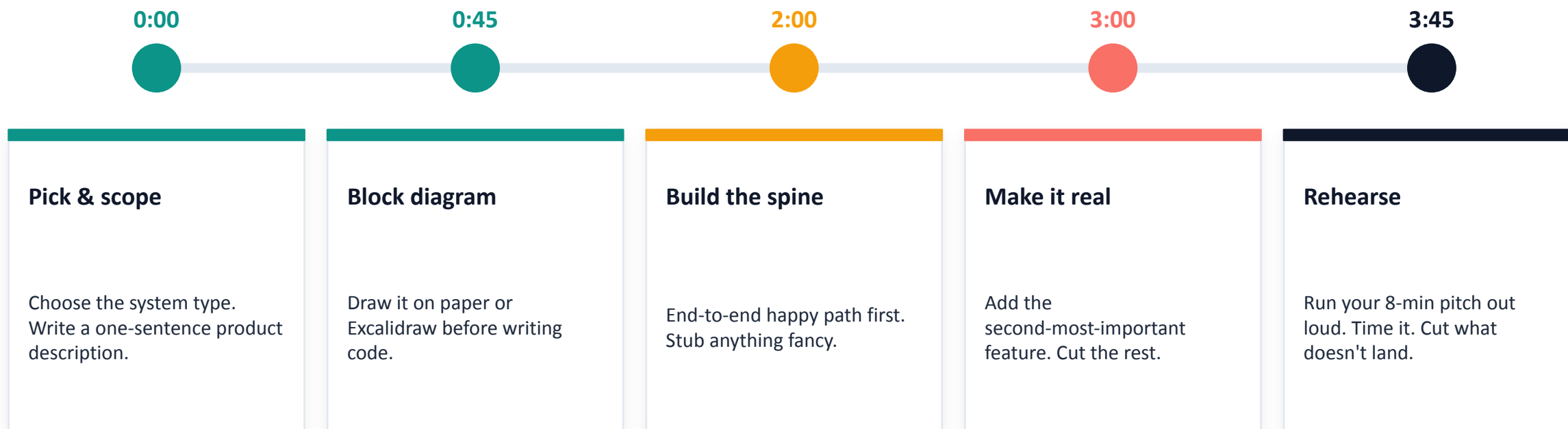
Hallucinating, looping, ignoring instructions — let's debug.



You want a dry run

Walk me through your 8-min pitch and I'll push back.

A suggested rhythm — not a rule



Pick a team. Pick a system. Let's build.